

第18章 EXTI—外部中断/事件控制器

本章参考资料：《STM32F4xx 中文参考手册》系统配置控制器以及中断和事件章节。

上一章我们已经详细介绍了 NVIC，对 STM32F4xx 中断管理系统有个全局的了解，我们这章的内容是 NVIC 的实例应用，也是 STM32F4xx 控制器非常重要的一个资源。学习本章时，配合《STM32F4xx 中文参考手册》系统配置控制器以及中断和事件章节一起阅读，效果会更佳，特别是涉及到寄存器说明的部分。

特别说明，本书内容是以 STM32F40xxx 系列控制器资源讲解。

18.1 EXTI 简介

外部中断/事件控制器(EXTI)管理了控制器的 23 个中断/事件线。每个中断/事件线都对应有一个边沿检测器，可以实现输入信号的上升沿检测和下降沿的检测。EXTI 可以实现对每个中断/事件线进行单独配置，可以单独配置为中断或者事件，以及触发事件的属性。

18.2 EXTI 功能框图

EXTI 的功能框图包含了 EXTI 最核心内容，掌握了功能框图，对 EXTI 就有一个整体的把握，在编程时就思路就非常清晰。EXTI 功能框图见图 18-1。

在图 18-1 可以看到很多在信号线上打一个斜杠并标注“23”字样，这个表示在控制器内部类似的信号线路有 23 个，这与 EXTI 总共有 23 个中断/事件线是吻合的。所以我们只要明白其中一个的原理，那其他 22 个线路原理也就知道了。

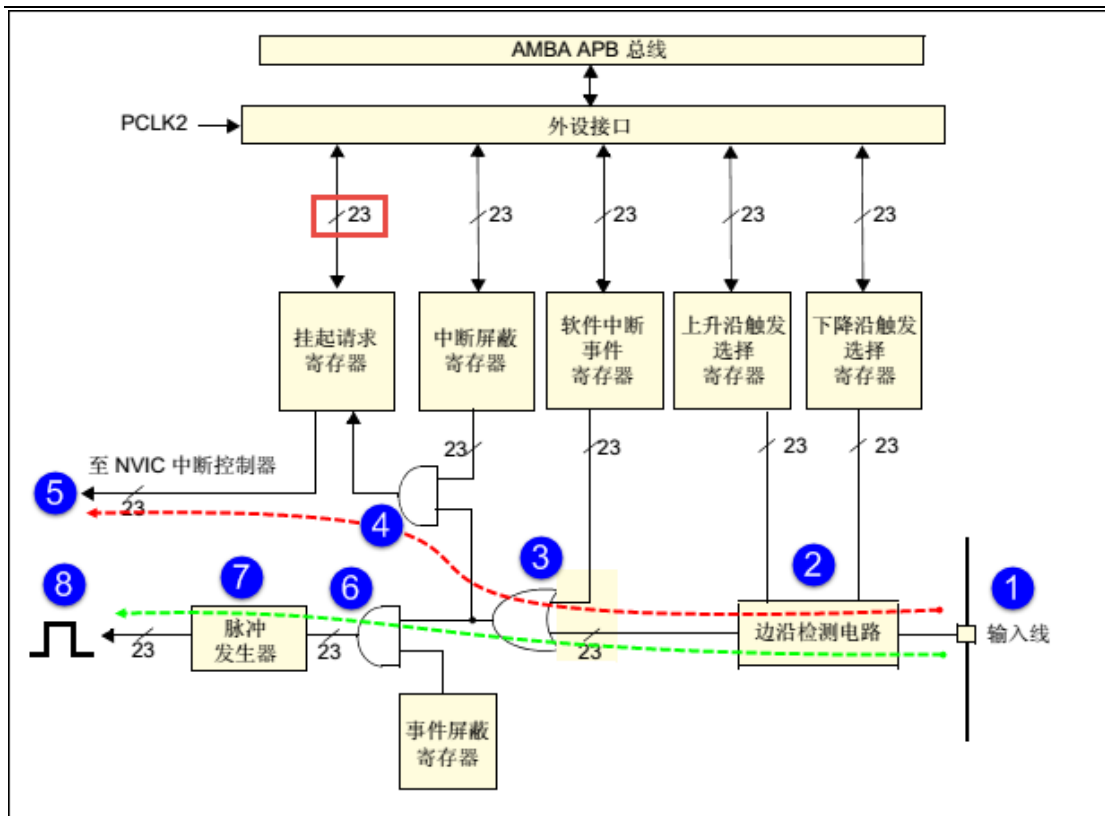


图 18-1 EXTI 功能框图

EXTI 可分为两大部分功能，一个是产生中断，另一个是产生事件，这两个功能从硬件上就有所不同。

首先我们来看图 18-1 中红色虚线指示的电路流程。它是一个产生中断的线路，最终信号流入到 NVIC 控制器内。

编号 1 是输入线，EXTI 控制器有 23 个中断/事件输入线，这些输入线可以通过寄存器设置为任意一个 GPIO，也可以是一些外设的事件，这部分内容我们将在后面专门讲解。输入线一般是存在电平变化的信号。

编号 2 是一个边沿检测电路，它会根据上升沿触发选择寄存器(EXTI_RTSR)和下降沿触发选择寄存器(EXTI_FTSR)对应位的设置来控制信号触发。边沿检测电路以输入线作为信号输入端，如果检测到有边沿跳变就输出有效信号 1 给编号 3 电路，否则输出无效信号 0。而 EXTI_RTSR 和 EXTI_FTSR 两个寄存器可以控制器需要检测哪些类型的电平跳变过程，可以是只有上升沿触发、只有下降沿触发或者上升沿和下降沿都触发。

编号 3 电路实际就是一个或门电路，它一个输入来自编号 2 电路，另外一输入来自软件中断事件寄存器(EXTI_SWIER)。EXTI_SWIER 允许我们通过程序控制就可以启动中断/事件线，这在某些地方非常有用。我们知道或门的作用就是有”就为 1，所以这两个输入随便一个有有效信号 1 就可以输出 1 给编号 4 和编号 6 电路。

编号 4 电路是一个与门电路，它一个输入编号 3 电路，另外一个输入来自中断屏蔽寄存器(EXTI_IMR)。与门电路要求输入都为 1 才输出 1，导致的结果如果 EXTI_IMR 设置为 0 时，那不管编号 3 电路的输出信号是 1 还是 0，最终编号 4 电路输出的信号都为 0；如果

EXTI_IMR 设置为 1 时，最终编号 4 电路输出的信号才由编号 3 电路的输出信号决定，这样我们可以简单的控制 EXTI_IMR 来实现是否产生中断的目的。编号 4 电路输出的信号会被保存到挂起寄存器(EXTI_PR)内，如果确定编号 4 电路输出为 1 就会把 EXTI_PR 对应位置 1。

编号 5 是将 EXTI_PR 寄存器内容输出到 NVIC 内，从而实现系统中断事件控制。

接下来我们来看看绿色虚线指示的电路流程。它是一个产生事件的线路，最终输出一个脉冲信号。

产生事件线路是在编号 3 电路之后与中断线路有所不同，之前电路都是共用的。编号 6 电路是一个与门，它一个输入编号 3 电路，另外一个输入来自事件屏蔽寄存器(EXTI_EMR)。如果 EXTI_EMR 设置为 0 时，那不管编号 3 电路的输出信号是 1 还是 0，最终编号 6 电路输出的信号都为 0；如果 EXTI_EMR 设置为 1 时，最终编号 6 电路输出的信号才由编号 3 电路的输出信号决定，这样我们可以简单的控制 EXTI_EMR 来实现是否产生事件的目的。

编号 7 是一个脉冲发生器电路，当它的输入端，即编号 6 电路的输出端，是一个有效信号 1 时就会产生一个脉冲；如果输入端是无效信号就不会输出脉冲。

编号 8 是一个脉冲信号，就是产生事件的线路最终的产物，这个脉冲信号可以给其他外设电路使用，比如定时器 TIM、模拟数字转换器 ADC 等等。

产生中断线路目的是把输入信号输入到 NVIC，进一步会运行中断服务函数，实现功能，这样是软件级的。而产生事件线路目的就是传输一个脉冲信号给其他外设使用，并且是电路级别的信号传输，属于硬件级的。

另外，EXTI 是在 APB2 总线上的，在编程时候需要注意到这点。

18.3 中断/事件线

EXTI 有 23 个中断/事件线，每个 GPIO 都可以被设置为输入线，占用 EXTI0 至 EXTI15，还有另外七根用于特定的外设事件，见表 18-1。

七根特定外设中断/事件线由外设触发，具体用法参考《STM32F4xx 中文参考手册》中对外设的具体说明。

表 18-1 EXTI 中断/事件线

中断/事件线	输入源
EXTI0	PX0 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI1	PX1 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI2	PX2 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI3	PX3 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI4	PX4 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI5	PX5 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI6	PX6 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI7	PX7 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI8	PX8 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI9	PX9 (X 可为 A, B, C, D, E, F, G, H, I)

EXTI10	PX10 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI11	PX11 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI12	PX12 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI13	PX13 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI14	PX14 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI15	PX15 (X 可为 A, B, C, D, E, F, G, H)
EXTI16	可编程电压检测器 (PVD) 输出
EXTI17	RTC 闹钟事件
EXTI18	USB OTG FS 唤醒事件
EXTI19	以太网唤醒事件
EXTI20	USB OTG HS (在 FS 中配置) 唤醒事件
EXTI21	RTC 入侵和时间戳事件
EXTI22	RTC 唤醒事件

EXTI0 至 EXTI15 用于 GPIO，通过编程控制可以实现任意一个 GPIO 作为 EXTI 的输入源。由表 18-1 可知，EXTI0 可以通过 SYSCFG 外部中断配置寄存器 1(SYSCFG_EXTICR1)的 EXTI0[3:0]位选择配置为 PA0、PB0、PC0、PD0、PE0、PF0、PG0、PH0 或者 PIO，见图 18-2。其他 EXTI 线(EXTI 中断/事件线)使用配置都是类似的。

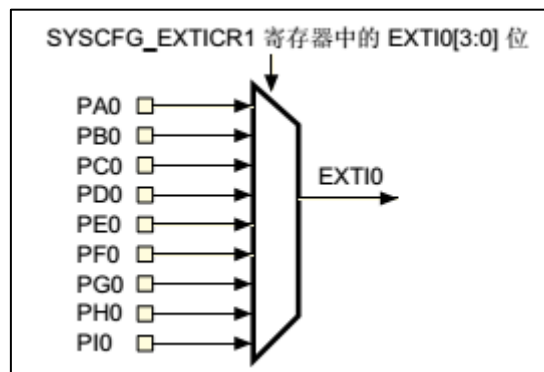


图 18-2 EXTI0 输入源选择

18.4 EXTI 初始化结构体详解

标准库函数对每个外设都建立了一个初始化结构体，比如 EXTI_InitTypeDef，结构体成员用于设置外设工作参数，并由外设初始化配置函数，比如 EXTI_Init()调用，这些设定参数将会设置外设相应的寄存器，达到配置外设工作环境的目的。

初始化结构体和初始化库函数配合使用是标准库精髓所在，理解了初始化结构体每个成员意义基本上就可以对该外设运用自如了。初始化结构体定义在 stm32f4xx_exti.h 文件中，初始化库函数定义在 stm32f4xx_exti.c 文件中，编程时我们可以结合这两个文件内注释使用。

代码清单 18-1 EXTI 初始化结构体

```

1 typedef struct {
2     uint32_t EXTI_Line;           // 中断/事件线
3     EXTI_Mode_TypeDef EXTI_Mode;  // EXTI 模式
4     EXTI_Trigger_TypeDef EXTI_Trigger; // 触发事件
5     FunctionalState EXTI_LineCmd; // EXTI 控制
6 } EXTI_InitTypeDef;
```

- 1) EXTI_Line: EXTI 中断/事件线选择, 可选 EXTI0 至 EXTI22, 可参考表 18-1 选择。
- 2) EXTI_Mode: EXTI 模式选择, 可选为产生中断(EXTI_Mode_Interrupt)或者产生事件(EXTI_Mode_Event)。
- 3) EXTI_Trigger: EXTI 边沿触发事件, 可选上升沿触发(EXTI_Trigger_Rising)、下降沿触发 (EXTI_Trigger_Falling) 或者上升沿和下降沿都触发 (EXTI_Trigger_Rising_Falling)。
- 4) EXTI_LineCmd: 控制是否使能 EXTI 线, 可选使能 EXTI 线(ENABLE)或禁用(DISABLE)。

18.5 外部中断控制实验

中断在嵌入式应用中占有非常重要的地位, 几乎每个控制器都有中断功能。中断对保证紧急事件得到第一时间处理是非常重要的

我们设计使用外接的按键来作为触发源, 使得控制器产生中断, 并在中断服务函数中实现控制 RGB 彩灯的任务。

18.5.1 硬件设计

轻触按键在按下时会使得引脚接通, 通过电路设计可以使得按下时产生电平变化, 见图 18-1。

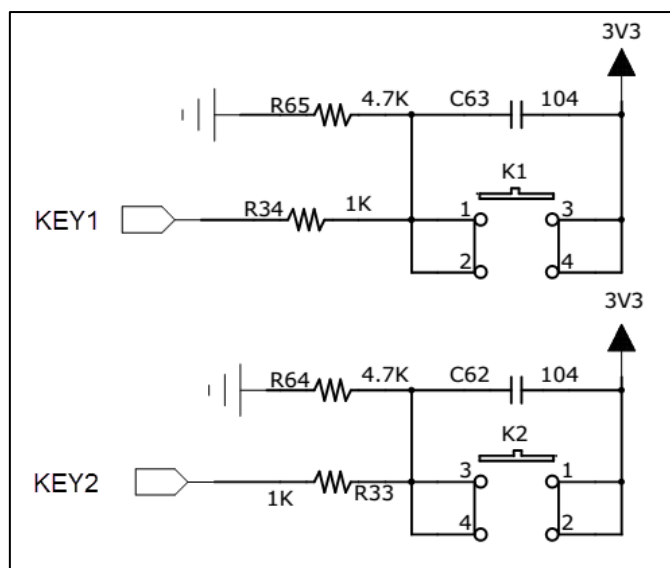


图 18-3 按键电路设计

18.5.2 软件设计

这里只讲解核心的部分代码，有些变量的设置，头文件的包含等并没有涉及到，完整的代码请参考本章配套的工程。我们创建了两个文件：bsp_exti.c 和 bsp_exti.h 文件用来存放 EXTI 驱动程序及相关宏定义，中断服务函数放在 stm32f4xx_it.h 文件中。

1. 编程要点

- 1) 初始化 RGB 彩灯的 GPIO；
- 2) 开启按键 GPIO 时钟和 SYSCFG 时钟；
- 3) 配置 NVIC；
- 4) 配置按键 GPIO 为输入模式；
- 5) 将按键 GPIO 连接到 EXTI 源输入；
- 6) 配置按键 EXTI 中断/事件线；
- 7) 编写 EXTI 中断服务函数。

2. 软件分析

按键和 EXTI 宏定义

代码清单 18-2 按键和 EXTI 宏定义

```
1 //引脚定义
2 /*****
3 #define KEY1_INT_GPIO_PORT          GPIOA
4 #define KEY1_INT_GPIO_CLK          RCC_AHB1Periph_GPIOA
5 #define KEY1_INT_GPIO_PIN          GPIO_Pin_0
6 #define KEY1_INT_EXTI_PORTSOURCE    EXTI_PortSourceGPIOA
7 #define KEY1_INT_EXTI_PINSOURCE    EXTI_PinSource0
8 #define KEY1_INT_EXTI_LINE          EXTI_Line0
9 #define KEY1_INT_EXTI_IRQ            EXTI0_IRQn
10
11 #define KEY1_IRQHandler              EXTI0_IRQHandler
12
13 #define KEY2_INT_GPIO_PORT          GPIOC
14 #define KEY2_INT_GPIO_CLK          RCC_AHB1Periph_GPIOC
15 #define KEY2_INT_GPIO_PIN          GPIO_Pin_13
16 #define KEY2_INT_EXTI_PORTSOURCE    EXTI_PortSourceGPIOC
17 #define KEY2_INT_EXTI_PINSOURCE    EXTI_PinSource13
18 #define KEY2_INT_EXTI_LINE          EXTI_Line13
19 #define KEY2_INT_EXTI_IRQ            EXTI15_10_IRQn
20
21 #define KEY2_IRQHandler              EXTI15_10_IRQHandler
```

使用宏定义方法指定与电路设计相关配置，这对于程序移植或升级非常有用。

嵌套向量中断控制器 NVIC 配置

代码清单 18-3 NVIC 配置

```
1 static void NVIC_Configuration(void)
2 {
3     NVIC_InitTypeDef NVIC_InitStructure;
4
5     /* 配置 NVIC 为优先级组 1 */
6     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_1);
7
8     /* 配置中断源: 按键 1 */
9     NVIC_InitStructure.NVIC_IRQChannel = KEY1_INT_EXTI_IRQ;
10    /* 配置抢占优先级: 1 */
11    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;
12    /* 配置子优先级: 1 */
13    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
14    /* 使能中断通道 */
15    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
16    NVIC_Init(&NVIC_InitStructure);
17
18    /* 配置中断源: 按键 2, 其他使用上面相关配置 */
19    NVIC_InitStructure.NVIC_IRQChannel = KEY2_INT_EXTI_IRQ;
20    NVIC_Init(&NVIC_InitStructure);
21 }
```

有关 NVIC 配置问题可参考上一章节内容, 这里不做过多解释。

EXTI 中断配置

代码清单 18-4 EXTI 中断配置

```
1 void EXTI_Key_Config(void)
2 {
3     GPIO_InitTypeDef GPIO_InitStructure;
4     EXTI_InitTypeDef EXTI_InitStructure;
5
6     /* 开启按键 GPIO 口的时钟 */
7     RCC_AHB1PeriphClockCmd(KEY1_INT_GPIO_CLK|KEY2_INT_GPIO_CLK, ENABLE);
8
9     /* 使能 SYSCFG 时钟, 使用 GPIO 外部中断时必须使能 SYSCFG 时钟 */
10    RCC_APB2PeriphClockCmd(RCC_APB2Periph_SYSCFG, ENABLE);
11
12    /* 配置 NVIC */
13    NVIC_Configuration();
14
15    /* 选择按键 1 的引脚 */
16    GPIO_InitStructure.GPIO_Pin = KEY1_INT_GPIO_PIN;
17    /* 设置引脚为输入模式 */
18    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN;
19    /* 设置引脚不上拉也不下拉 */
20    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
21    /* 使用上面的结构体初始化按键 */
22    GPIO_Init(KEY1_INT_GPIO_PORT, &GPIO_InitStructure);
23
24    /* 连接 EXTI 中断源 到 key1 引脚 */
25    SYSCFG_EXTILineConfig(KEY1_INT_EXTI_PORTSOURCE,
26                          KEY1_INT_EXTI_PINSOURCE);
27
28    /* 选择 EXTI 中断源 */
29    EXTI_InitStructure.EXTI_Line = KEY1_INT_EXTI_LINE;
30    /* 中断模式 */
31    EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
```



```
32  /* 下降沿触发 */
33  EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Rising;
34  /* 使能中断/事件线 */
35  EXTI_InitStructure.EXTI_LineCmd = ENABLE;
36  EXTI_Init(&EXTI_InitStructure);
37
38  /* 选择按键 2 的引脚 */
39  GPIO_InitStructure.GPIO_Pin = KEY2_INT_GPIO_PIN;
40  /* 其他配置与上面相同 */
41  GPIO_Init(KEY2_INT_GPIO_PORT, &GPIO_InitStructure);
42
43  /* 连接 EXTI 中断源 到 key2 引脚 */
44  SYSCFG_EXTILineConfig(KEY2_INT_EXTI_PORTSOURCE,
45                        KEY2_INT_EXTI_PINSOURCE);
46
47  /* 选择 EXTI 中断源 */
48  EXTI_InitStructure.EXTI_Line = KEY2_INT_EXTI_LINE;
49  EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
50  /* 上升沿触发 */
51  EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Falling;
52  EXTI_InitStructure.EXTI_LineCmd = ENABLE;
53  EXTI_Init(&EXTI_InitStructure);
54 }
```

首先，使用 `GPIO_InitTypeDef` 和 `EXTI_InitTypeDef` 结构体定义两个用于 GPIO 和 EXTI 初始化配置的变量，关于这两个结构体前面都已经做了详细的讲解。

使用 GPIO 之前必须开启 GPIO 端口的时钟；用到 EXTI 必须开启 SYSCFG 时钟。

调用 `NVIC_Configuration` 函数完成对按键 1、按键 2 优先级配置并使能中断通道。

作为中断/时间输入线把 GPIO 配置为输入模式，这里不使用上拉或下拉，有外部电路完全决定引脚的状态。

`SYSCFG_EXTILineConfig` 函数用来指定中断/事件线的输入源，它实际是设定 SYSCFG 外部中断配置寄存器的值，该函数接收两个参数，第一个参数指定 GPIO 端口源，第二个参数为选择对应 GPIO 引脚源编号。

我们的目的是产生中断，执行中断服务函数，EXTI 选择中断模式，按键 1 使用下降沿触发方式，并使能 EXTI 线。

按键 2 基本上采用与按键 1 相关参数配置，只是改为上升沿触发方式。

EXTI 中断服务函数

代码清单 18-5 EXTI 中断服务函数

```
1 void KEY1_IRQHandler(void)
2 {
3     //确保是否产生了 EXTI Line 中断
4     if (EXTI_GetITStatus(KEY1_INT_EXTI_LINE) != RESET) {
5         // LED1 取反
6         LED1_TOGGLE;
7         //清除中断标志位
8         EXTI_ClearITPendingBit(KEY1_INT_EXTI_LINE);
9     }
10 }
11
12 void KEY2_IRQHandler(void)
13 {
14     //确保是否产生了 EXTI Line 中断
15     if (EXTI_GetITStatus(KEY2_INT_EXTI_LINE) != RESET) {
```



```
16      // LED2 取反
17      LED2_TOGGLE;
18      //清除中断标志位
19      EXTI_ClearITPendingBit(KEY2_INT_EXTI_LINE);
20  }
21 }
```

当中断发生时，对应的中断服务函数就会被执行，我们可以在中断服务函数实现一些控制。

一般为确保中断确实发生，我们会在中断服务函数调用中断标志位状态读取函数读取外设中断标志位并判断标志位状态。

EXTI_GetITStatus 函数用来获取 EXTI 的中断标志位状态，如果 EXTI 线有中断发生函数返回“SET”否则返回“RESET”。实际上，EXTI_GetITStatus 函数是通过读取 EXTI_PR 寄存器值来判断 EXTI 线状态的。

按键 1 的中断服务函数我们让 LED1 翻转其状态，按键 2 的中断服务函数我们让 LED2 翻转其状态。执行任务后需要调用 EXTI_ClearITPendingBit 函数清除 EXTI 线的中断标志位。

主函数

代码清单 18-6 主函数

```
1 int main(void)
2 {
3     /* LED 端口初始化 */
4     LED_GPIO_Config();
5
6     /* 初始化 EXTI 中断，按下按键会触发中断，
7      * 触发中断会进入 stm32f4xx_it.c 文件中的函数
8      * KEY1_IRQHandler 和 KEY2_IRQHandler，处理中断，反转 LED 灯。
9      */
10    EXTI_Key_Config();
11
12    /* 等待中断，由于使用中断方式，CPU 不用轮询按键 */
13    while (1) {
14    }
15 }
```

主函数非常简单，只有两个任务函数。LED_GPIO_Config 函数定义在 bsp_led.c 文件内，完成 RGB 彩灯的 GPIO 初始化配置。EXTI_Key_Config 函数完成两个按键的 GPIO 和 EXTI 配置。

18.5.3 下载验证

保证开发板相关硬件连接正确，把编译好的程序下载到开发板。此时 RGB 彩色灯是暗的，如果我们按下开发板上的按键 1，RGB 彩灯变亮，再按下按键 1，RGB 彩灯又变暗；如果我们按下开发板上的按键 2 并弹开，RGB 彩灯变亮，再按下开发板上的 KEY2 并弹开，RGB 彩灯又变暗。